

(1) 3.3.2

For this exercise, we provided the AI with the core architectural documentation for our Flask application, JobVago. The materials included the functional requirements, data dictionary, class diagram, and the comprehensive 278-case test suite. We then prompted it to act as a senior python backend developer and selected the `AuthenticationService` for this exercise because its heavily documented business logic and strict boundary conditions made it an ideal candidate to test the AI's precision. We made a prompt that synthesized the class signature, functional requirements security constraints, and the exact expected error strings from our 15 test cases.

The AI's initial generation was logically sound but architecturally imprecise. It violated the separation of concerns by attempting to hash passwords directly in the service layer rather than delegating to the `User` model, and it mistakenly applied whitespace stripping *after* string length validation. We provided a targeted follow-up prompt to enforce sequential validation and proper model abstraction.

After the single iteration, the generated code successfully passed all 15 Pytest cases on the first run. This exercise highlighted that while AI excels at translating constraints into functional code, it requires explicit architectural steering. For future generation tasks, feeding the AI the raw test files directly would likely eliminate ambiguity and reduce the need for manual architectural corrections.

(2)3.3.3

We provided the same resources as in 3.3.2 and requested from it to critically evaluate the codebase quality based on the test documentation and propose structural improvements. We then tried to get it to carry out some of its suggested changes

The AI accurately identified a vulnerability in the system's maintainability: the high cyclomatic complexity ($V(G)=9$) within the `CourseService._apply_filters` method, which was evident from the Basis Path testing documentation. We subsequently prompted the AI to execute a refactoring of this specific module.

The outcome significantly exceeded my expectations. The AI successfully reverse-engineered the likely monolithic `if/elif` structure and replaced it with a highly extensible Filter Pipeline utilizing the Strategy Pattern. This adhered to the Open-Closed Principle and cleanly separated the filtering logic from the core service layer, fully satisfying the non-functional maintainability requirements (NFR3).

While the AI was highly capable of inferring the code structure from the test cases, an extended iteration was required because the source code (`jobvago.zip`) failed to upload correctly. To increase the efficiency and precision of future interactions, We will ensure that explicit code snippets are provided alongside the testing documentation to eliminate the need for the AI to reverse-engineer the initial state. Ultimately, the AI proved to be a highly effective pair-programming partner for architectural review and refactoring.